

# A Teacher-Anchored Residual Calibration Framework for Drift-Adaptive GKP Decoding

Draft research note

May 29, 2026

## Abstract

Approximate Gottesman–Kitaev–Preskill (GKP) quantum error correction encodes a qubit into an oscillator and uses continuous-valued modular syndromes to infer displacement errors. In realistic finite-energy GKP states, syndrome statistics are shaped by state-preparation width, measurement noise, circuit-level imperfections, and slow calibration drift. This note develops a two-timescale decoding framework for such settings. A low-latency fast loop applies an FPGA-friendly affine correction  $\Delta_t = K_t s_t + b_t$ , while a slower calibration loop estimates the current effective noise state from recent syndrome histograms and updates the runtime parameters. The learned component is deliberately narrow: a lightweight CNN is anchored by a classical teacher estimator and predicts only a bounded residual calibration term, currently focused on  $b_t$ , rather than replacing the GKP decoder end-to-end.

The method is positioned against four nearby bodies of work: analog soft-information decoding for bosonic/GKP codes, syndrome-statistics and calibration-based decoder-prior adaptation, learned low-latency QEC decoders, and real-time FPGA decoding systems. The intended contribution is therefore not a new maximum-likelihood GKP decoder or a board-level FPGA demonstration. It is a deployment-constrained calibration layer whose per-shot path remains a fixed-point affine rule, while the slow loop performs teacher-anchored residual adaptation under explicit clipping, smoothing, stage-and-commit, and software-HIL evidence boundaries.

## Contents

|          |                                                                    |          |
|----------|--------------------------------------------------------------------|----------|
| <b>1</b> | <b>Scope and Evidence Boundary</b>                                 | <b>2</b> |
| <b>2</b> | <b>Introduction</b>                                                | <b>2</b> |
| <b>3</b> | <b>Summary of Contributions</b>                                    | <b>3</b> |
| <b>4</b> | <b>Brief Review of the GKP Code</b>                                | <b>3</b> |
| 4.1      | Ideal and approximate code states . . . . .                        | 3        |
| 4.2      | Syndrome measurement as modular displacement information . . . . . | 4        |
| 4.3      | Local affine decoding and its limits . . . . .                     | 4        |
| 4.4      | Logical failure criterion . . . . .                                | 4        |
| <b>5</b> | <b>Noise and Drift Model</b>                                       | <b>5</b> |
| 5.1      | Effective noise state . . . . .                                    | 5        |
| 5.2      | Noise sources represented by the model . . . . .                   | 5        |
| 5.3      | Drift scenarios . . . . .                                          | 5        |

|          |                                                              |          |
|----------|--------------------------------------------------------------|----------|
| <b>6</b> | <b>Model Architecture</b>                                    | <b>5</b> |
| 6.1      | Fast loop: affine fixed-point decoder . . . . .              | 5        |
| 6.2      | Parameter mapping . . . . .                                  | 6        |
| 6.3      | Teacher estimators . . . . .                                 | 6        |
| 6.4      | CNN residual branch . . . . .                                | 6        |
| 6.5      | Stage-and-commit runtime contract . . . . .                  | 7        |
| <b>7</b> | <b>Relationship to Existing Work</b>                         | <b>7</b> |
| 7.1      | GKP and bosonic analog soft information . . . . .            | 7        |
| 7.2      | Adaptive priors and syndrome-statistics estimation . . . . . | 7        |
| 7.3      | Learned low-latency QEC modules . . . . .                    | 7        |
| 7.4      | Real-time and FPGA QEC decoders . . . . .                    | 8        |
| <b>8</b> | <b>Numerical Results and Benchmark Plan</b>                  | <b>8</b> |
| <b>9</b> | <b>Writing Position and Near-Term Plan</b>                   | <b>8</b> |

# 1 Scope and Evidence Boundary

## 2 Introduction

Bosonic codes protect quantum information by encoding a logical qubit into the Hilbert space of an oscillator. The GKP code is a central example: ideal code states form a periodic lattice in phase space, so small shift errors can be diagnosed through modular syndrome measurements and corrected by displacement operations [1, 2]. This structure is attractive because the measured syndrome is analog rather than merely binary; it carries information about both the likely displacement and the confidence of the correction.

The practical challenge is that physical GKP states are approximate finite-energy states. Their comb peaks have finite width and an envelope, and syndrome measurements are further affected by measurement inefficiency, noisy ancilla states, circuit-level imperfections, oscillator loss, and calibration error [2, 8]. Consequently, the effective noise model seen by the decoder can change over time. A fixed syndrome-to-correction map may be well tuned for one operating point but mismatched after bias, variance, or covariance drift.

Recent GKP and bosonic-decoding work makes clear that analog soft information is valuable. Surface-GKP and concatenated bosonic-QLDPC decoders use continuous syndrome information to set matching weights, belief-propagation messages, or effective outer-code priors [4, 5, 6, 7]. At the same time, recent QEC systems papers show that useful learned modules tend to be narrow, modular, and latency-aware rather than unconstrained replacements for the full decoder [13, 14]. Hardware and calibration oriented work similarly emphasizes that decoder priors must be kept aligned with device statistics [9, 10, 11, 12].

This project applies that systems principle at the physical GKP correction layer. The fast loop is intentionally simple:

$$\Delta_t = K_t s_t + b_t, \tag{1}$$

where  $s_t$  is the measured two-quadrature GKP syndrome and  $(K_t, b_t)$  are quantized runtime parameters. The slow loop observes recent syndrome histograms, estimates an effective noise state, and updates  $(K_t, b_t)$  at a lower rate. The CNN is not asked to learn the entire GKP decoder. It is anchored by a classical teacher estimator and learns a bounded residual calibration term.

The resulting claim is deliberately narrow: this is a teacher-anchored residual calibration layer for a deployment-constrained affine GKP fast path. That narrower formulation is important because several broad claims would be unsafe. The project is not the first use of GKP analog syndrome information, not the first adaptive decoder-prior idea, not the first calibration-conditioned neural QEC decoder, and not the first real-time FPGA QEC decoder. Its niche is the combination of GKP physical-layer affine correction, histogram-driven drift adaptation, teacher-residual learning, and explicit runtime contracts.

### 3 Summary of Contributions

The target paper can be organized around three contributions.

**1. A two-timescale adaptive affine formulation for approximate GKP decoding.** We formulate the per-shot correction path as a bounded affine estimator  $\Delta_t = K_t s_t + b_t$ . The expensive adaptation problem is moved into a slow loop that estimates the current effective noise state from recent syndrome statistics. This links the continuous-syndrome GKP picture to a runtime contract that can be implemented as a quantized matrix-vector operation.

**2. A teacher-anchored residual calibration strategy.** The mainline uses a classical teacher estimator to produce a stable baseline  $(K_t^{\text{teacher}}, b_t^{\text{teacher}})$ . The CNN predicts a small residual, currently

$$\delta b_t = f_\phi(H_{t-c+1:t}, \Delta H_t, z_t^{\text{teacher}}), \quad (2)$$

and the committed parameters are

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}(b_t^{\text{teacher}} + \delta b_t). \quad (3)$$

This treats learning as calibration of a low-dimensional control surface, not as a full neural GKP decoder.

**3. A deployment-aware runtime architecture.** The architecture exposes the constraints that matter for real-time use: fixed-point quantization, clipping, saturation diagnostics, parameter smoothing, stale-parameter behavior, and double-buffered stage-and-commit updates. These constraints become measurable objects in the benchmark, rather than prose assumptions after the algorithm is designed.

## 4 Brief Review of the GKP Code

### 4.1 Ideal and approximate code states

The square-lattice GKP code encodes a qubit into an oscillator by using periodic structure in the  $q$  and  $p$  quadratures [1]. In the project convention, the lattice constant is

$$\lambda = \sqrt{2\pi}. \quad (4)$$

An ideal logical state may be represented heuristically as an infinite comb, for example

$$|\bar{0}\rangle \propto \sum_{n \in \mathbb{Z}} |n\lambda\rangle_q. \quad (5)$$

Ideal comb states are not physical because they require infinite energy. Approximate GKP states replace the infinitely sharp comb with finitely squeezed peaks and an envelope. The PRX Quantum review of the GKP code emphasizes that finite-energy structure is central to practical GKP QEC rather than a minor implementation detail [2]. It determines the intrinsic syndrome uncertainty and contributes an effective displacement-like noise.

## 4.2 Syndrome measurement as modular displacement information

Let the accumulated phase-space displacement error before correction be

$$e_t = \begin{bmatrix} e_{q,t} \\ e_{p,t} \end{bmatrix}. \quad (6)$$

An ideal GKP syndrome observes this displacement modulo the lattice:

$$s_t = e_t \bmod \lambda, \quad s_{q,t}, s_{p,t} \in [-\lambda/2, \lambda/2). \quad (7)$$

In a finite-energy and noisy-measurement setting, the measured syndrome is more accurately described as

$$\tilde{s}_t = \text{mod}(e_t, \lambda) + \eta_t^{\text{meas}} + \eta_t^{\text{GKP}}. \quad (8)$$

Here  $\eta_t^{\text{meas}}$  represents measurement and ancilla contributions, while  $\eta_t^{\text{GKP}}$  summarizes finite-squeezing uncertainty. The decoder therefore observes a noisy modular representative, not the absolute displacement.

## 4.3 Local affine decoding and its limits

The modulo structure makes exact GKP decoding nonlinear and branch dependent. Near a single lattice branch, however, a Gaussian approximation gives a useful local model. If  $e$  and  $s$  are jointly Gaussian,

$$\begin{bmatrix} e \\ s \end{bmatrix} \sim \mathcal{N} \left( \begin{bmatrix} \mu_e \\ \mu_s \end{bmatrix}, \begin{bmatrix} \Sigma_{ee} & \Sigma_{es} \\ \Sigma_{se} & \Sigma_{ss} \end{bmatrix} \right), \quad (9)$$

then the linear-MMSE estimate is

$$\hat{e} = \mu_e + \Sigma_{es} \Sigma_{ss}^{-1} (s - \mu_s) = Ks + b, \quad (10)$$

where

$$K = \Sigma_{es} \Sigma_{ss}^{-1}, \quad b = \mu_e - K \mu_s. \quad (11)$$

This derivation explains why an affine fast path is a plausible low-latency approximation. It also marks the boundary of the approximation. Near lattice decision boundaries, the posterior can be multimodal; maximum-likelihood, closest-lattice-point, or wrapped-Gaussian decoders may outperform a single global affine rule [3, 8].

## 4.4 Logical failure criterion

After correction, the residual displacement is wrapped into the GKP fundamental cell. A logical error occurs when the residual crosses a logical decision boundary, for example

$$|r_{q,t}| > \lambda/2 \quad \Rightarrow \quad X_L \text{ error}, \quad (12)$$

and similarly for the  $p$ -quadrature. This is why closed-loop logical error rate, or the repository's `final_ler_mean` proxy under a fixed protocol, is more important than offline parameter-regression error.

## 5 Noise and Drift Model

### 5.1 Effective noise state

The slow loop does not attempt to infer every microscopic hardware parameter. It uses a low-dimensional effective state,

$$\theta_t^{\text{noise}} = (\sigma_t, \mu_{q,t}, \mu_{p,t}, \vartheta_t), \quad (13)$$

where  $\sigma_t$  controls the displacement scale,  $\mu_{q,t}$  and  $\mu_{p,t}$  represent mean bias, and  $\vartheta_t$  captures covariance-axis rotation. This is a deliberately compact parameterization: it can be mapped to affine runtime parameters and estimated from finite histogram windows.

### 5.2 Noise sources represented by the model

The present model is best understood as an effective displacement-noise model with additional measurement uncertainty. It is meant to absorb several physical effects:

1. finite-energy GKP peak width and envelope effects;
2. Gaussian random displacement noise in  $q$  and  $p$ ;
3. biased displacement means caused by calibration offsets;
4. anisotropic or rotated covariance caused by asymmetric noise;
5. syndrome measurement noise and noisy auxiliary-state contributions.

This is narrower than a full circuit-level GKP noise model, but it is aligned with the current affine fast-path contract.

### 5.3 Drift scenarios

The current frozen software-HIL protocol uses four named drift scenarios:

1. `static_bias_theta`: a biased and rotated operating point;
2. `linear_ramp`: slowly changing effective noise parameters;
3. `step_sigma_theta`: a sudden variance/orientation change;
4. `periodic_drift`: periodic nonstationarity.

These scenarios are useful because they separate different adaptation demands: steady calibration, tracking, shock response, and periodic following. A full paper should add unseen families such as random-walk drift, burst measurement noise, coupled bias-variance drift, faster-than-window drift, and low-shot histogram regimes.

## 6 Model Architecture

### 6.1 Fast loop: affine fixed-point decoder

The fast loop receives the current measured syndrome  $s_t$ , reads the active parameter bank, and computes

$$s_t^{\text{clip}} = \text{clip}(s_t, -s_{\text{max}}, s_{\text{max}}), \quad (14)$$

$$\Delta_t^{\text{raw}} = K_t s_t^{\text{clip}} + b_t, \quad (15)$$

$$\Delta_t = Q(\text{clip}(\Delta_t^{\text{raw}}, -\Delta_{\text{max}}, \Delta_{\text{max}})), \quad (16)$$

where  $Q(\cdot)$  denotes fixed-point quantization. The project convention is an FPGA-oriented Q4.20 representation for syndrome values and runtime parameters. The fast loop also records diagnostic counters, including histogram-input saturation, correction saturation, overflow, aggressive parameter events, commit counts, and fallback-related signals.

## 6.2 Parameter mapping

Given an estimated noise state, the parameter mapper constructs an error covariance

$$C = R(\vartheta) \begin{bmatrix} \sigma_q^2 & 0 \\ 0 & \sigma_p^2 \end{bmatrix} R(\vartheta)^\top, \quad (17)$$

and a measurement covariance

$$R_{\text{meas}} = (\sigma_{\text{meas}}^2 + \Delta_{\text{eff}}^2)I. \quad (18)$$

The raw gain is

$$K_{\text{raw}} = C(C + R_{\text{meas}})^{-1}, \quad (19)$$

followed by gain clipping or scaling. The bias target is

$$b_{\text{target}} = \alpha(I - K_{\text{target}})\mu, \quad \mu = \begin{bmatrix} \mu_q \\ \mu_p \end{bmatrix}. \quad (20)$$

Finally, exponential smoothing produces staged parameters:

$$K_t = (1 - \beta)K_{t-1} + \beta K_{\text{target}}, \quad b_t = (1 - \beta)b_{t-1} + \beta b_{\text{target}}. \quad (21)$$

## 6.3 Teacher estimators

The teacher family provides stable estimates of  $\theta_t^{\text{noise}}$  from recent syndrome history. A simple teacher computes moments over a histogram window. Stronger teachers such as EKF, UKF, RLS, or particle-filter variants add state-space assumptions over time. Abstractly,

$$\hat{\theta}_t^{\text{teacher}} = \text{Teacher}(H_{1:t}), \quad (22)$$

and

$$(K_t^{\text{teacher}}, b_t^{\text{teacher}}) = \text{ParamMapper}(\hat{\theta}_t^{\text{teacher}}). \quad (23)$$

The teacher is the stability anchor. It keeps the learned branch near an interpretable calibration surface and creates meaningful ablation baselines.

## 6.4 CNN residual branch

The CNN consumes a short context of normalized syndrome histograms and histogram deltas:

$$X_t^{\text{hist}} = [H_{t-c+1}, \dots, H_t, \Delta H_{t-c+2}, \dots, \Delta H_t]. \quad (24)$$

Teacher-side features may include

$$z_t^{\text{teacher}} = (\hat{\theta}_t^{\text{teacher}}, K_t^{\text{teacher}}, b_t^{\text{teacher}}, \Delta b_t^{\text{teacher}}, \dots). \quad (25)$$

The current gated branch narrows this to a small set of teacher- $b$  and teacher- $\Delta b$  scalars. The learned branch predicts

$$\widehat{\delta b}_t = f_\phi(X_t^{\text{hist}}, z_t^{\text{teacher}}), \quad (26)$$

then clips the result:

$$\delta b_t = \text{clip}(s_b \widehat{\delta b}_t, -b_{\max}, b_{\max}). \quad (27)$$

The committed parameter is

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}(b_t^{\text{teacher}} + \delta b_t). \quad (28)$$

## 6.5 Stage-and-commit runtime contract

The slow loop does not directly mutate active fast-loop parameters. It stages a candidate  $(K, b)$  into an inactive bank and commits it at a safe epoch boundary:

$$(K_t, b_t) = \begin{cases} (K^A, b^A), & t < t_{\text{commit}}, \\ (K^B, b^B), & t \geq t_{\text{commit}}. \end{cases} \quad (29)$$

This contract allows the benchmark to measure stale-parameter effects, update latency, commit success, rollback/fallback behavior, and fixed-point stability.

# 7 Relationship to Existing Work

## 7.1 GKP and bosonic analog soft information

GKP analog syndrome information has already been shown to be valuable. Work on analog QEC, surface-GKP codes, and concatenated bosonic-QLDPC decoding uses continuous measurement information to set decoder weights or soft messages [3, 4, 5, 6, 7]. This project should not claim novelty for using analog information itself. Its distinction is that the analog statistics are compressed into a physical-layer affine GKP fast-path parameter update.

## 7.2 Adaptive priors and syndrome-statistics estimation

Adaptive-weight and syndrome-statistics methods estimate decoder weights or noise parameters from measured data [9, 10]. Calibrated decoders and decoder-prior optimization show that prior mismatch can materially affect logical performance [11, 12]. This project borrows that lesson, but maps the adaptation target to the GKP affine parameters  $(K, b)$  and evaluates it under a stage-and-commit runtime contract.

## 7.3 Learned low-latency QEC modules

AI pre-decoders and calibration-conditioned FiLM decoders demonstrate a systems-oriented pattern: a learned module is useful when its input/output contract, latency role, and integration with a classical decoder are explicit [13, 14]. This project follows the same philosophy, but its learned module is slower than the per-shot path and updates only calibration parameters. The per-shot path remains affine fixed-point logic.

## 7.4 Real-time and FPGA QEC decoders

Real-time QEC decoders and FPGA-integrated systems have already reported low-latency hardware decoding in stabilizer-code settings [15, 16]. Those works define the standard for true deployment claims: closed-loop timing, worst-case latency, resource use, and hardware integration. The present project is not yet at that evidence level. The safe near-term claim is FPGA-friendly architecture and software-HIL validation, not FPGA demonstration.

## 8 Numerical Results and Benchmark Plan

## 9 Writing Position and Near-Term Plan

## References

- [1] D. Gottesman, A. Kitaev, and J. Preskill, “Encoding a qubit in an oscillator,” *Physical Review A*, 2001.
- [2] A. L. Grimsmo and S. Puri, “Quantum Error Correction with the Gottesman–Kitaev–Preskill Code,” *PRX Quantum* 2, 020101, 2021.
- [3] K. Fukui, A. Tomita, A. Okamoto, and K. Fujii, “High-threshold fault-tolerant quantum computation with analog quantum error correction,” *Physical Review X*, 2018.
- [4] K. Noh and C. Chamberland, “Fault-tolerant bosonic quantum error correction with the surface-GKP code,” *Physical Review A*, 2020.
- [5] K. Noh, C. Chamberland, and F. G. S. L. Brandao, “Low-overhead fault-tolerant quantum error correction with the surface-GKP code,” *PRX Quantum*, 2022.
- [6] L. Berent et al., “Analog information decoding of bosonic quantum LDPC codes,” *PRX Quantum*, 2024.
- [7] S. K. Borah, A. K. Pradhan, N. Raveendran, M. Pacenti, and B. Vasic, “Fault Tolerant Decoding of QLDPC-GKP Codes with Circuit Level Soft Information,” preprint, 2025.
- [8] M.-A. Roy, T. Pousset, and B. Royer, “Decoding Multimode Gottesman–Kitaev–Preskill Codes with Noisy Auxiliary States,” preprint, 2025.
- [9] S. Spitz, B. Tarasinski, C. W. J. Beenakker, and T. E. O’Brien, “Adaptive weight estimator for quantum error correction,” *Advanced Quantum Technologies*, 2018.
- [10] T. Wagner, H. Kampermann, and D. Bruß, “Optimal noise estimation from syndrome statistics of quantum codes,” *Physical Review Research*, 2021.
- [11] E. H. Chen et al., “Calibrated decoders for experimental quantum error correction,” *Physical Review Letters*, 2022.
- [12] V. Sivak, M. Newman, and P. Klimov, “Optimization of decoder priors for accurate quantum error correction,” *Physical Review Letters* 133, 150603, 2024.
- [13] C. Chamberland, J. Olle, M. Li, S. Thornton, and I. Baratta, “Fast and accurate AI-based pre-decoders for surface codes,” arXiv:2604.12841, 2026.

- [14] S. Stein, S. Kan, C. Liu, A. Harkness, S. Garner, Z. Du, Y. Ding, Y. Mao, and A. Li, “Calibration-Conditioned FiLM Decoders for Low-Latency Decoding of Quantum Error Correction Evaluated on IBM Repetition-Code Experiments,” preprint, 2026.
- [15] X. Yang et al., “Real-time Surface-Code Error Correction Using an FPGA-based Neural-Network Decoder,” preprint, 2026.
- [16] L. Caune et al., “Demonstrating real-time and low-latency quantum error correction with superconducting qubits,” preprint, 2024.